Exam Preparation

•Networking (Week 7)

•IAM (Week 5)

•Miscellaneous (Week5 + Week 7 + Lab 5)

•AI (Week 8 + Labs 8&9)

•**DevOps (Week 10 + Lab 7)**

•Miscellaneous (Week7 + Week 9 + Lab 6)

EC2 instance: (short for Elastic Compute Cloud instance) is a AWS service that provides scalable computing capacity in the cloud.

API Gateway: handle requests, forward requests to backend

IAM

**An organisation has 5 departments and has separated out each of the IAM users into separate groups using paths following the pattern companybucket/department1/, *companybucket /department2/*, companybucket/department3/* etc.

Their IAM account names follow the pattern user@department1.company.com, user@department2.company.com etc.

You are tasked with securing a bucket that contains a folder for each of 5 departments in an organisation. Only people within a department can write to their own folder. Everyone can read from all folders.

Discuss the principles that you would use to create a policy that would achieve this objective.

Write the policy as a JSON file that you would use.**

Principles to consider:

1. **Least Privilege** : Grant users only the permissions they need to perform their tasks. In this case, each department's users need read access to all folders and write access only to their department's folder.
2. **Resource-based policies** : Define policies that restrict access based on the IAM paths and folder names in the bucket.
3. **Granular Access Control** : Ensure that permissions are specific enough to allow specific access based on the bucket's folder structure.
4. **Path-based Permissions** : Use path-based folder structures in the S3 bucket to restrict write permissions based on the user's department.
5. **Read-Only Access for All** : Grant read permissions to all departments across the entire bucket to support the requirement that everyone can read from all folders.


```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowReadFromAllFolders",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::companybucket/*"
    },
    {
      "Sid": "AllowWriteToDepartment1Folder",
      "Effect": "Allow",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::companybucket/department1/*",
      "Condition": {
        "StringLike": {
          "aws:username": "*@department1.company.com"
        }
      }
    },
```

Network

When an EC2 instance is created in AWS, it is assigned to a region and a Virtual Private Cloud (VPC). Describe how network addresses are allocated to a VPC and sub-networked when an EC2 instance is created. How is the EC2 connected to other machines and to the Internet?

How network addresses are allocated to a VPC:

- Assign the VPC a CIDR block , such as `10.0.0.0/16 (65,536 IP addresses)`, which defines the *IP address range* for all resources within the VPC.
- This CIDR block is then divided into subnets.

How sub-networked when an EC2 instance is created:

- **Subnets** are created within a VPC, each assigned a specific portion of the VPC's IP range. For example, a VPC with a CIDR block of `10.0.0.0/16` could have subnets like:
  - `10.0.1.0/24` for one subnet (providing 256 addresses),
  - `10.0.2.0/24` for another subnet, and so forth.
- Subnets can be designated as **public** or **private** :
  - **Public Subnets** : Have Internet access via an **Internet Gateway (IGW)** .
  - **Private Subnets** : No direct Internet access; use **NAT Gateway** for outbound connectivity if needed.
- (IP allocation) When an EC2 instance is launched within a subnet, it is assigned:
  - **Private IP** : Each EC2 instance gets a private IP within its subnet.

    - **Public IP** : Optionally assigned in a public subnet for Internet access; static **Elastic IP** can be used for consistent access.

To connect to other machines:

- **Route Tables** : Each subnet is associated with a route table that defines the traffic flow rules. Route tables have entries specifying how packets are routed within the VPC, between subnets, and externally.
- **Security Groups and Network ACLs** : Both **Security Groups** (instance-level firewall) and **Network Access Control Lists (NACLs)** (subnet-level firewall) define access rules for inbound and outbound traffic.

Access the main **route table** to know the reachable addresses of other machines in the VPC.

- if address is in local VPC, it can connect directly or through the router if the machine is in a different subnet.
- if address is not in local VPC, it can connect through the internet gateway according to NACLs.

To connect to other networks:

- **Internet Gateway (IGW)** : Provides public Internet access for instances in public subnets.
- **NAT Gateway** : Enables private instances to access the Internet without direct exposure.

Describe 2 ways in which security is implemented in AWS networks at the network level. What are the similarities and differences between these 2 security implementations?

2 ways of implementing security: security group and network access control lists.

Similarities:

- Both control **inbound and outbound traffic** and use IP addresses and port rules.
- Both are components of AWS **VPC security** and provide an additional layer of network protection.

security group: it controls the traffic that is allowed to reach and leave the resources at the instance level.

1. support rules only.
2. stateful: If an inbound request is allowed, the response is automatically allowed.
3. instance-level

- Use Case : Commonly used for controlling access to individual instances, such as allowing SSH or HTTP access.

Network ACLs (network access control lists): it allows or denies specific traffic to reach and leave the resources at subnet level.

1. support and deny rules.
2. stateless: Each inbound and outbound rule must be explicitly defined; no automatic responses are allowed.
3. subnet level

- Use Case : Used for broader control of subnet traffic, such as blocking specific IP ranges from accessing an entire subnet.

ALB (load balancing)

Steps to create an ALB for EC2 instances in two different availability zones:

1. create and configure security group by authorising inbound traffic for SSH and HTTP.
2. get availability zones in the region
3. create a key pair for future ssh connection
4. create EC2 instances by specifying instance type, security group and availability zones
5. get available subnets in the region
6. create a load balancer by specifying name, subnets, security group, scheme (internet-facing or internal), IP address type (IPv4 or dualstack).
7. create and configure target group by specifying name, protocol (HTTP), port (80), vpc id, port to use for health check, target type.
8. register targets for the configured target group.
9. create and configure listener by specifying load balancer arn, protocol, port, and action (forward HTTP requests to the target group)

Steps to test the ALB:

1. ssh into EC2 instance (establish secure shell connection)
2. update instance and install apache2 (web server software)
3. edited HTML tags of the default Apache web page
4. accessed instance by their public IP address in browser

Steps to set up load balancer to Django application:

1. create an EC2 instance by specifying instance type, security group and availability zones, and ssh into the instance
2. install virtual environment package, set up a virtual environment, activate the virtual environment
3. install django, create a django project and create a django app
4. install nginx and configure nginx, restart nginx
5. setup the django app (register the app in settings.py, update views.py and urls.py)
6. (enable template rendering and render templates with retrieved data)
7. run the web server and access the app through the EC2 instance's public IP address
8. create an ALB by specifying name, subnets, security group, scheme (internet-facing or internal), IP address type (IPv4 or dualstack).
9. create and configure target group by specifying name, protocol to use (HTTP), port to forward traffic (80), vpc id, port to use for health check, health check interval, health check path, target type.
10. register targets for the configured target group
11. create and configure listener by specifying load balancer arn, protocol, port, and action (forward HTTP requests to the target group)

ML/AI

- Underfitting: model is too “simple” to represent all the relevant class characteristics
    - High bias and low variance
    - High training error and high test error
- Overfitting: model is too “complex” and fits irrelevant characteristics (noise) in the data
    - Low bias and high variance

- Low training error and high test error

Describe the 3 different types of Machine Learning Models that AWS Machine Learning allows you to use and describe examples of the types of questions you could answer with each one.

https://docs.aws.amazon.com/machine-learning/latest/dg/types-of-ml-models.html

Binary Classification Model

"Is this email spam or not spam?"

"Will the customer buy this product?"

"Is this product a book or a farm animal?"

"Is this review written by a customer or a robot?"

Multiclass Classification Model

"Is this product a book, movie, or clothing?"

"Is this movie a romantic comedy, documentary, or thriller?"

"Which category of products is most interesting to this customer?"

Regression Model

"What will the temperature be in Seattle tomorrow?"

"For this product, how many units will sell?"

"What price will this house sell for?"

Steps for using SageMaker for Machine Learning:

1. collect data and formulate the problem (input and target)
2. prepare a sagemaker session:
    - get the SageMaker execution role ARN
    - create a S3 bucket for holding input and output
3. prepare dataset:
    - collect labelled data and setup a target variable and a list of feature variables
    - preprocess data (data cleaning, data transform, feature scaling: create categorical variables, convert categorical variables to dummy variables, drop columns, convert boolean columns to numeric)
    - split into training, validation and test dataset
    - upload train and validation datasets to S3 bucket
4. select a machine learning model:
    - classification: logistic regression, decision tree, random forest, XGBoost, neural network
    - regression: linear regression, regression tree, XGBoost

5. set up tuning job name and configuration (hyperparameters ranges, tuning strategy, total number of training jobs, max number of parallel jobs, metrics used for tuning)
6. specify the machine learning algorithm used for tuning job (retrieve training image, specify S3 paths for datasets, define input data for training and validation channels, specify output location, define compute resou1rces, specify IAM role, static hyperparameters for the algorithm, stopping condition)
7. launch the tuning job: train the model using training set and tune hyperparameters using validation set
8. evaluate the model on test dataset, use metrics like AUC score, f1, precision, recall
9. check if the result support the theory/achieve our goal

How to evaluate the model:

- Select evaluation metrics:
    - Binary classification – Accuracy (balanced dataset), Precision, recall, F1, AUC
    - Multiclass classification – Precision, Recall, F1, AUC
    - Regression – RMSE, Accuracy
- Compare the results of training and testing set under each evaluation metric to see if any overfitting issues exist
- Compare the results of the selected model and a single variable model under each evaluation metric to see if the selected model has a good performance.

Entities: pieces of information extracted from texts. They are *objects, concepts, or categories* such as places, organisations, names, dates, and quantities. For example, "Sam was born in France," "Sam" is a person, "France" is location. These are entities in this sentence.

Keyphrases: meaningful *pieces of text* that capture the *main ideas* of a paragraph or article. They are generally nouns, verbs or phrases that summarise key concepts or central topics in the text. For example, in a sentence about cats' daily routine, the key phrases might be "cat", "sleep", "eat", "play" and "grooming".

Syntaxes: the rules of how words are arranged to form sentences. It is about the structure of sentences and determines how different parts of speech, e.g. nouns, verbs, pronouns and adjectives combine to convey clear messages. For example, in the sentence "The cat is cute", "cat" is the subject, "is" is the verb, "cute" is the adjective. This sentence follows the basic structure "subject + linking verb + predicate adjective".

How to use Rekognition:

- create S3 bucket and upload images to it
- perform operations like detect_labels, detect_moderation_labels, detect_faces, detect_text.

s3.upload_file(Filename, Bucket, Key): (local file path, bucket name, destination path in S3)

Architecture

Describe the different ways in which a Microservice Architecture can be implemented on AWS and what the benefits of this approach might be. How could authentication and authorisation be handled in this approach?

Replicate web-based software functionalities:

- use ec2 instances to host the web interface, handling user requests and managing sessions.
- S3 bucket can store [items like videos] to manage retention and cost
- use Lambda functions to trigger processing events (when a user upload a video) [e.g. converting video formats and resolutions]. It scales automatically with usage and reduces the need for always-on servers
- use DynamoDB to store accounts [e.g. social media accounts]
- use ELB to distribute network traffic to the EC2 instances
- use CloudWatch to monitor performance metrics for services and logging
- API Gateway routes requests to Lambda

DevOps

What is DevOps and describe how you would implement the automation of creation of machines, configuration of software and deployment of application programs using AWS.

What is DevOps:

1. Creating automation for the entire build/test/release cycle,
2. allows Development, Test and Production environments to be built, tested and deployed easily and systematically,
3. Allows new components to be added automatically,
4. Could be done manually with the help of scripts but there are software environments that manage remote communication and execution of resource creation, software installation and configuration.

How to implement:

1. Using fabric to connect to an SSH server and executing commands remotely.
2. Using the version control tool AWS CodeCommit to maintain changes in the source code.
3. Using AWS CodePipeline to perform Continuous Integration/Continuous Delivery (CI & CD) service for updating code.
4. Using AWS CodeBuild to compile source code, run tests, and produce software packages.
5. Using AWS CodeDeploy to test and deploy new code.
6. Using AWS CloudWatch to automate monitoring and logging, enabling alerting, error detection, and performance insights for rapid issue resolution.
7. Use ELB to automatically adjust resources based on demand, maintaining reliability and availability.
8. Use AWS IAM for access control, use Secrets Manager for managing credentials, and enforce least-privilege access across resources.

Use Fabric to automate the setup of a Python 3 virtual environment, nginx and a Django app within the EC2 instance we created:

1. create and configure a security group
2. create an EC2 instance
3. install Fabric and create a config file in ~/.ssh
4. connect to the EC2 instance and automate the setup of Python3 virtual env, nginx, and a Django app within the EC2 instance
5. install virtual environment package, set up a virtual environment, activate the virtual environment
6. install django, create a django project and create a django app
7. install nginx and configure nginx, restart nginx
8. edit Django files (views.py and urls.py)

9. run the web server and access the app through the EC2 instance's public IP address

Serverless Architecture:

Compute:

- AWS Lambda : Executes video processing tasks on-demand, ideal for infrequent use.

Storage:

- Amazon S3 : Stores videos and metadata, integrates with Lambda, and optimises storage costs.

API Management:

- API Gateway : Exposes a REST API to trigger Lambda functions for user requests.

Database:

- DynamoDB/Aurora Serverless : Scalable storage for metadata or user data, handling varying usage efficiently.

Pros and Cons of serverless services (lambda, S3):

pros:

- **Cost Efficiency** : Pay only for actual use, ideal for predictable workloads.
- **Scalability** : scale automatically with demand.
- **Reduced Overhead** : AWS manages infrastructure, allowing focus on development.
- **Reliability** : increase fault tolerance and high availability.

cons:

- **Cold Start Latency** : Occasional delays for inactive functions.
- **Execution Limits** : 15-minute limit on Lambda tasks.
- **Workflow Complexity** : Event-driven setups can be harder to debug.
- **Limited Control** : Less customisation than EC2-managed environments.

AWS Services General

Question:

A software developer wants to create a Drobox-like application for Photographers where they could seamlessly upload pictures from a local machine to the cloud and synchronise the data across multiple devices (Computer, Mobile Phones, etc.) for authenticated users. For every image uploaded by the users, she wants to save the metadata related to the file. Given the files uploaded by users are very important, she wants to allow restoring such files even if users delete the images on purpose. In addition, the software developer wants to automatically create labels for the uploaded images so they can perform searches of images based on the generated labels.

Describe your approach using AWS technologies to create that application.

- use EC2 instances to host web services that provides REST API for processing user requests and interacting with other system components

- use Amazon S3 to store [images uploaded by users], which provides scalabel and durable storage. Each user can have a unique S3 bucket to organise their files
- use Lambda function to trigger events each time a new [file: image] is uploaded. The event triggers processing tasks like [meta data extraction and image labelling]
- use DynamoDB to store [metadata and labels for each image]
- use Amazon Rekognition to process images and do labelling. Each time an image is uploaded, an S3 event triggers a Lambda function that calls Amazon Rekognition to generate labels.
- use CloudTrail to track actions and changes [made by users, such as uploads, downloads, or deletions]
- use IAM to authenticate users and give specific permission to specific users.
    - S3 access: allow each user to upload, view, and delete files in their S3 directory
    - DynamoDB access: enable users to manage their [image metadata]

Questions:

A MedTech company in Perth has created a novel algorithm for detecting heart diseases from X-rays and CT scans (medical images) using machine learning and computer vision. The company now wants to commercialise it as a Software-as-a-service (SaaS) product. Initially, this product should allow authorised radiologist working at the Royal Perth Hospital to upload such images. Then, once the algorithm can process the image (it could take a couple of minutes) it should retrieve the prediction. The solution should be scalable so it can be used in other hospitals in the future.

Give 10 recommendations to the company on how they could use AWS to create this platform. Provide your answer as a list of bullet points.

- use S3 to store image files
- use IAM to control access and permissions of users.
- use SageMaker to train machine learning models [heart disease detection model] using these images and make predictions
- use EC2 instances to host web services that provides REST API for processing user requests and interacting with other system components
- use ELB to distribute network traffic to the EC2 instances
- use Lambda function to trigger events that notifies the system every time an image is uploaded
- use CloudWatch for monitoring and logging: monitor application performance, prediction latency, user activity, setting up alerts for failures or anomalies
- Use AWS KMS for managing encryption keys, ensuring compliance with [medical data protection regulations]
- use DynamoDB to store [image metadata and prediction results]

Cloud Services pros and cons

Provide 5 pros and 5 cons of moving infrastructure from being on-premises to cloud based. Consider this in the context of global expansion and resilience. Provide your answer as a list of bullet points.

pros

1. cloud-based systems have *higher scalability*, compared to data centres.
2. cost of using and maintaining a cloud based infrastructure are much *cheaper* than data centres, pay as you go, *no upfront investment*

3. lower latency: given that cloud are distributed aboud the globe, services can operate faster in Asia and China for an Australian based company
4. more *resource-saving* compare to data centres since it scales according to demand
5. cloud are *easy to recover* from incidents and easy to back up as compared to data centres

cons:

1. *local government regulations* may not support cloud based technologies (China donot support their own data stored on clouds that are based outside of China)
2. data centre are more *secure* than using clouds, given that data centre are fully owned by the company, and therefore data will be stored fully by the company, whereas for cloud based, data have to upload to clouds, where data are exposed to leakage, especially when using dockers.
3. data centres can achieve more *customised* services compared to cloud, given that data centres are fully owned by the company, whereas cloud based can only use the services provided by the cloud.
4. change on to cloud based is challenging for *accounting* departments
5. once a cloud service is chosen (e.g. google cloud or AWS) it is *hard to shift to other* cloud or their own databse if the company find the services provided is unsatiisfying

Docker and container

A container is a standard unit of software that packages up the application and all its dependencies and libraries by virtualising the operating system.

Docker offers the ability to run multiple applications within the same host OS, sharing underlying resources.

Docker vs Container:

1. Similarity: Both containers and docker images allow users to specify application dependencies and configurations and to describe everything necessary for a machine to run the application.
2. Differences: Docker is a platform for managing containers, while a container is the isolated environment where an application runs. All Containers use the same underlying OS but present various parts of the OS as if they were dedicated to the running system.

Docker vs VMBox:

1. Similarity: Both Docker and VirtualBox are virtualisation tools running on the computer.
2. Differences: VirtualBox is an virtualization app that create virtual machines that are *isolated at the hardware level*, whereas Docker is a containerization app that *isolates apps at application level*.